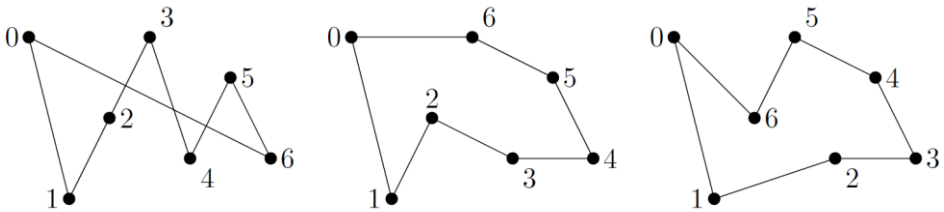
DM – Recherche d'un plus court chemin

1. PRESENTATION

On s'intéresse au problème du voyageur de commerce qui doit visiter **N villes** une seule fois puis revenir à son point de départ. On cherche à déterminer **le chemin le plus court possible**. La figure suivante représentent trois chemins différents possibles pour un problème à **7** villes, les numéros indiquent l'ordre de passage dans chaque ville (**0** représente donc le point de départ et d'arrivée).



Nous allons essayer de trouver une solution à ce problème pour un ensemble de **58 villes en France**. Pour cela, on dispose d'un fichier texte **Villes.txt** qui contient **58 lignes** de la forme :

Nom de la ville , latitude , longitude , population

Les latitude et longitude sont données en degrés.

2. QUELQUES FONCTIONS UTILES

Dans cette partie, nous allons tout d'abord écrire quelques fonctions qui seront utilisées dans les parties suivantes pour résoudre le problème.

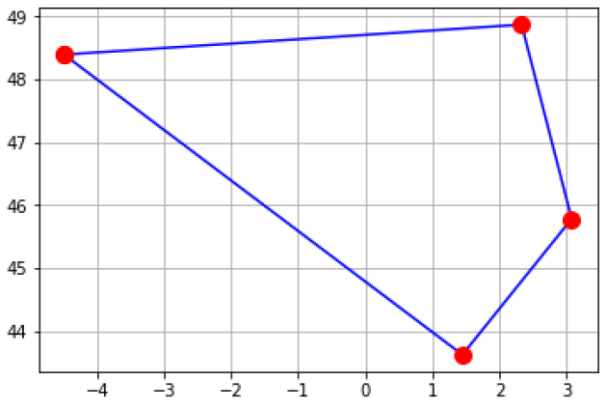
On veut tout d'abord écrire une fonction **ChargerVilles()** qui lit le fichier **Villes.txt** et qui retourne une liste **VILLES** dont chacun des **58 éléments** est une liste contenant le nom (**str**), la latitude (**float**), la longitude (**float**) et la population (**int**) d'une ville. On montre l'utilisation de cette fonction en console et le début de ce qui est affiché ci-dessous :

```
In [2]: ChargerVilles()
Out[2]:
[['Lorient', 47.7504, -3.3666, 84952],
 ['Brest', 48.3904, -4.495, 144899],
 ['Aix-en-Provence', 43.52, 5.45, 146821],
 ['Ajaccio', 41.9271, 8.7283, 54364],
```

Q1) Ecrire la fonction **ChargerVilles()**. Vous pourrez utiliser la méthode **split** sur une chaîne de caractères pour séparer une chaîne de caractères en une liste de chaînes de caractères. Vous prendrez soin de bien convertir les coordonnées en **float**.

Un parcours sera représenté par une liste d'entiers représentant le numéro de la ville visitée (son index dans la liste **VILLES**). Par exemple, le parcours **P = [2,0,4,1]** représente un chemin qui commence par la ville **2**, puis la ville **0**, … pour finir par revenir à la liste **2** de départ. Le retour à la ville de départ est implicite, il n'apparaît pas dans la liste **P**.

Q2) Après avoir importé le module **matplotlib.pyplot** avec son alias **plt**, écrire une fonction **AfficheParcours(VILLES, P)** qui affiche un parcours **P** (longitude en abscisse et latitude en ordonnée. Un exemple de tracé pour le parcours **P1 = [1,8,9,17]** est donné ci-contre :



La distance **D** entre deux points A_1 et A_2 de la surface de la terre définis par leurs longitudes ψ_1 et ψ_2 et leurs latitudes φ_1 et φ_2 est : $D = R \arccos\left(\sin(\varphi_1)\sin(\varphi_2)+\cos(\varphi_1)\cos(\varphi_2)\cos(\psi_1-\psi_2)\right)$ avec $R = 6370,7\ km$.

Q3) Ecrire une fonction **Distance(A1,A2)** qui prend en paramètres deux points et qui retourne la distance entre ces deux points. **A1** et **A2** sont deux listes de la forme **[latitude, longitude]** (en **degrés**).

Q4) Ecrire une fonction **DistanceParcours(VILLES,P)** qui prend en paramètre la liste **VILLES** et un parcours **P** et qui retourne la distance totale de ce parcours. N'oubliez pas que le chemin doit revenir à son point de départ !

Par exemple, pour le parcours **P1 = [1,8,9,17]**, **DistanceParcours(VILLES,P1)** retourne **1825,8 km**.

3. LA METHODE « BOURRIN »

Il est maintenant de temps de rechercher le chemin le plus court. Nous allons commencer par essayer la méthode "bourrin" et tester tous les chemins possibles.

Q5) Pour un problème de **N villes**, combien de parcours différents possibles existent-ils (on considère comme identiques deux parcours effectués dans un sens et dans le sens contraire) ?

Q6) En considérant que le temps de calcul nécessaires à l'évaluation d'un chemin prend **1ns**, déterminer le nombre maximum N_{MAX} de villes qu'on pourrait prendre en compte avec ce type de méthode si on veut que le temps de calcul total ne dépasse pas une minute.

Pour énumérer toutes les permutations possibles des villes, on peut utiliser un algorithme récursif qui utilise l'idée suivante : les permutations d'une liste **L** de longueurs **n** sont obtenues en prenant les **n** valeurs possibles de la dernière valeur (**L[-1]**) précédées de toutes les permutations possibles des autres éléments de **L**. Ce principe est utilisé dans la fonction **Affiche_Permutations(k,L)** ci-dessous qui affiche la liste des permutations des **k** premiers éléments d'une liste (donc en exécutant **Affiche_Permutations(len(L),L)** on affiche toutes les permutations possibles de **L**).

```
def Affiche_Permut(k,L):
    if k==1:
        print(L)
    else:
        for i in range(k):
            L[k-1],L[i]=L[i],L[k-1]
            Affiche_Permut(k-1,L)
            L[k-1],L[i]=L[i],L[k-1]
```

Q7) Modifier la fonction précédente pour écrire une fonction **Recherche_Meilleur_Parcours_BOURRIN(k,L,VILLES)** qui utilisent des variables globales (à cause de la récursivité !) pour mémoriser le meilleur chemin exploré et sa longueur. Cette fonction sera utilisée par une fonction **Meilleur_Parcours_BOURRIN(n,VILLES)** donnée ci-dessous qui renvoie le meilleur parcours et sa distance pour les **n** premières villes de la liste **VILLES**.

```
def Meilleur_Parcours_BOURRIN(n,VILLES):
    global D_min,P
    P=[k for k in range(n)]
    D_min=DistanceParcours(VILLES,P)
    Recherche_Meilleur_Parcours_BOURRIN(n,P,VILLES)
    return P,D_min
```

Q8) Essayer de tester cette méthode pour les **6** premières villes (**n=6**). Vous devriez trouver le meilleur parcours **[0, 4, 3, 2, 5, 1] avec 2548 km**. Essayer ensuite pour **n=10**. Conclure sur cette méthode.

4. UNE METHODE UTILISABLE POUR LES 58 VILLES

Pour déterminer le chemin le plus court, on se propose d'utiliser une autre méthode. Une première idée consiste à faire subir une modification aléatoire (une permutation de deux villes choisies de manière aléatoire) à un chemin de départ et de la garder si cette modification réduit la distance puis de recommencer. Cependant, on peut rapidement converger ainsi vers un minimum local et être très loin du minimum général. Il faut donc aussi permettre des échanges qui augmente la longueur du chemin pour espérer la réduire par la suite. Cette possibilité devra être offerte surtout

au début du processus puis de moins en moins au fur et à mesure de la recherche. C'est pourquoi on l'utilisera avec une probabilité qui devra être importante au début mais décroître au fur et à mesure. L'algorithme utilisé sera le suivant :

```
Initialiser le parcours P à une valeur quelconque
POUR i allant de 0 à M-1 FAIRE
        D1 = Distance du parcours P
        Modifier P en échangeant deux villes aléatoirement
        D2 = Distance du parcours modifié
        SI D2 < D1 ALORS
                Conserver la modification
        SINON
                Conserver la modification avec une probabilité p
        FIN SI
FIN POUR
```

La probabilité **p** devra évoluer au cours de l'algorithme en fonction de :

- la différence de distance **D2-D1** : plus la différence de distance est grande, moins la probabilité de conserver la modification doit être importante.
- Le numéro **i** de l'itération : la probabilité de conserver la modification doit décroitre au fur et à mesure de l'itération.

On choisit donc de calculer la probabilité **p** selon la relation suivante :

$$\mathbf{p(D1,D2,i) = e^{-\frac{D2-D1}{f(i)}}}$$ avec $\mathbf{f}$ une fonction décroissante telle que $\mathbf{f(0) = f_{max}}$ et $\mathbf{f(M) = 0}$.

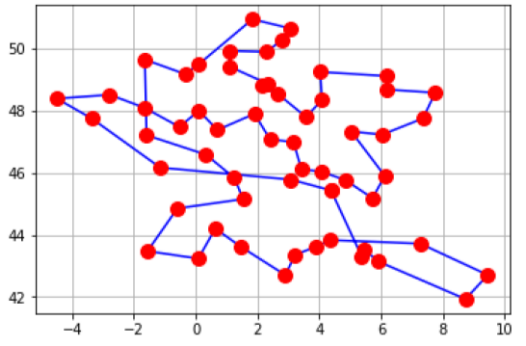
Q9) Ecrire une fonction **f(i,M,fmax)** qui retourne **f(i)**. On pourra choisir dans un premier temps une fonction linéaire puis expérimenter d'autres fonctions.

Q10) Ecrire une fonction **p(D1,D2,i,M,fmax)** qui retourne la probabilité **p** défini comme précédemment.

Q11) Ecrire une fonction **Echange(P,i,j)** qui échange les villes d'indices **i** et **j** du parcours **P**.

Q12) Ecrire une fonction **Meilleur_Parcours_2(VILLES,M,fmax)** qui utilise l'algorithme décrit précédemment et retourne le meilleur parcours et sa distance. Pour tirer un entier au hasard entre a et b, on peut utiliser la fonction **randint(a,b)** du module **random**.

Q13) Essayer cette fonction pour différentes valeurs de **M** et **fmax**. On pourra écrire un programme qui utilise cette fonction pour différentes valeurs de **M** et **fmax** et ne conserve que le meilleur résultat. De cette manière, j'ai pu trouver un parcours reliant les **58** villes de longueur **6673 km** (avec **M=25000** , **fmax=100** avec une loi linéaire). Avec un peu de chance, vous ferez mieux !



1 parcours
de 6673 km.

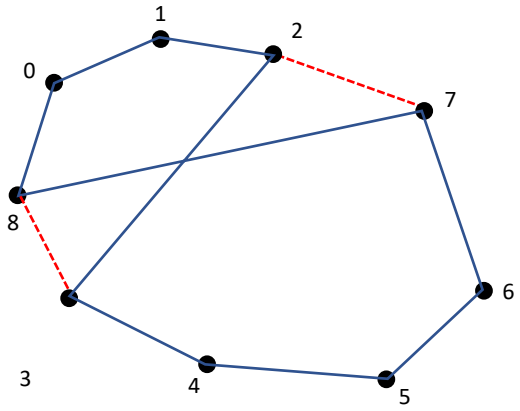
5. AMELIORATION DE LA SOLUTION

La méthode précédente permet de trouver les parcours « pas trop longs » passant par les 58 villes. Mais on peut sans doute faire mieux.

ESSAIS DE TOUS LES ECHANGES POSSIBLES

Une première méthode est d'examiner à partir de la solution obtenue précédemment s'il existe un échange de villes qui améliore la solution. Pour cela, on crée une fonction **OptimisationEchange(VILLES,P)** qui prend en argument un parcours, teste tous les échanges de villes possibles et retourne le meilleur parcours trouvé avec sa distance.

Q14) Ecrire la fonction **OptimisationEchange(VILLES,P)**.



SUPPRESSION DES CROISEMENTS

Les parcours proposés par les méthodes précédentes peuvent encore comporter des segments qui se croisent et on peut les optimiser en supprimant ces croisements. Par exemple, l'exemple ci-contre, montre un croisement entre les segments $[2,3]$ et $[7,8]$ sur le parcours initial. On peut supprimer ce croisement en créant les liaisons en pointillés $[2,7]$ et $[3,8]$ et en parcourant la partie $[3,4,5,6,7]$ du chemin initial en sens inverse.

Soit une ville d'indice **i** dans un parcours **P**, on note **S(i)** l'indice de la ville suivante dans le parcours. **S(i)=i+1** sauf pour la dernière ville du parcours pour laquelle **S(i)=0**. On considère un parcours **P** pour lequel la liaison $[i,S(i)]$ et la liaison $[j,S(j)]$ se croisent. Pour supprimer ce croisement, on crée un nouveau parcours en conservant le parcours initial jusqu'à l'indice **i**, puis on parcourt la liaison $[i,j]$, puis de **j** à $S(i)$ (portion du parcours initial en sens inverse), puis la liaison $[S(i),S(j)]$, puis la fin du parcours est inchangée.

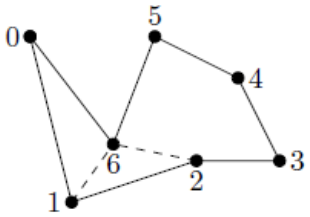
Q15) Ecrire une fonction **SuppressionCroisement(P,i,j)** qui retourne le nouveau parcours après suppression du croisement sans modifier le parcours initial.

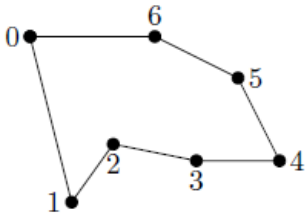
Une solution serait de rechercher les croisements dans un parcours pour les supprimer. Mais la recherche de ces croisements est assez complexe et demande du temps d'exécution. Une méthode plus simple à programmer et qui ne prend pas beaucoup plus de temps et d'examiner toutes les paires (i,j) avec $i<j$ possibles et de ne conserver que les changements qui réduisent la longueur du parcours.

Q16) Ecrire une fonction **OptimisationCroisement(VILLES,P)** qui prend en paramètre un parcours **P** et qui effectue les changements de parcours décrits ci-dessus. Cette fonction renverra le nouveau parcours et sa longueur.

INTEGRER UNE VILLE DANS UN SEGMENT

Nous allons introduire une dernière modification de parcourt qui pourrait faire gagner quelques kilomètres. Les parcours renvoyés pourraient ressembler à celui-ci-dessous (à gauche). Dans ce cas, il est clair que la ville numéro **6** doit être intégrée au segment $[1,2]$ pour réduire la longueur totale.





Q17) Ecrire une fonction **IntégrerVilleSegment(P,i,j)** qui retourne un nouveau parcours (sans modifier l'ancien) en intégrant le point d'indice **i** au segment $[j,S(j)]$.

Comme précédemment, on procède de manière brutale en testant l'intégration de toutes les villes du parcours à tous les segments en ne gardant que les modifications qui réduisent la longueur du chemin.

Q18) Ecrire une fonction **OptimisationIV(VILLES,P)** qui prend en paramètre un parcours **P** et qui effectue les changements de parcours décrits ci-dessus. Cette fonction renverra le nouveau parcours et sa longueur.

Q19) Il ne vous reste plus qu'à utiliser tous ces outils pour trouver le meilleur parcours. Le tableau ci-dessous vous donne une idée de la qualité du parcours trouvé :

Longueur (km)	Appréciation
20000	Nul ! Un enfant de 4 ans fait mieux !
10000	Encore trop mauvais !
7000	Ça commence à ressembler à quelque-chose, vous êtes sur la bonne voie !
6500	Bien, on se rapproche d'une vraie solution !
6000	Très bien !
5800	Excellent !
5788,4	Mon record ! Si vous faîtes mieux, vous avez droit à mes félicitations … et à un petit cadeau !